

TUT-8

gets :- funⁿ that reads
a string from
standard input

int *

Q1) void remov(char arr[], char a);

int main(void)

{

char arr[20];

printf("Enter a string: ");

gets(arr);

remov(arr, 'o');

return 0;

}

void remov(char arr[], char a)

{

char *b, arr2[20];

int i = 0;

```
for (b = arr; *b != '\0'; b++)
```

```
{  
    if (*b == a)
```

```
        continue;
```

```
    else
```

```
        *(arr2 + i++) = *b;
```

```
}
```

```
* (arr2 + i++) = '\0';
```

```
strcpy (arr, arr2);
```

```
printf ("String after removal of given character is: ");
```

```
puts (arr2);
```

```
return;
```

```
}
```

Q2) int main ()

```
{
```

```
    int x, y, *a, *b, temp;
```

```
    printf ("Enter the value of x and y")
```

```
    scanf ("%d %d", &x, &y);
```

```
    printf ("Before swapping = %d by = %d \n", x, y);
```

```
    a = &x;
```

```
    b = &y;
```

```
    *temp = *a;
```

```
    *a = *b;
```

```
    *b = *temp;
```

```
print ("After swapping = %d \n y = %d \n", x, y);
```

```
return 0;
}
```

```
1) int main()
{
    int a[3][4] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};
    printf ("%u, %u, %u \n", a[0]+1, *(a[0]+1), *(a[0]+1));
    return 0;
}
```

array begins at 1002

Q3)

```
a) int main()
```

```
{
```

```
int i=3, *j, k;
```

```
j = &i;
```

```
printf ("%d \n", i**j*i + *j); → 3 × 3 × 3 + 3 = 30
```

```
return 0;
```

```
}
```

OUTPUT

1006, 2, 2

```
b) int main()
```

```
{
```

```
int x=30, *y, *z;
```

```
y = &x;
```

```
z = y;
```

```
*y++ = *z++;
```

```
x++;
```

```
printf ("x = %d, y = %d, z = %d \n", x, y, z);
```

```
return 0;
```

```
}
```

Ans:-

x = 31

y = address of z

z = address of y

```
1) int main()
```

```
{
    int arr[3] = {2, 3, 4};
```

```
char *p;
```

```
p = arr;
```

```
p = (char*)((int*)(p));
```

```
printf ("%d", *p);
```

```
p = (int*)(p+1);
```

```
printf ("%d", *p);
```

```
return 0;
```

```
}
```

because of lack of &*char to work

OUTPUT

2, 0

```
12) int main()
```

```
{
    char *str
```

```
str = "%d/n";
```

```
str++;
```

```
str++;
```

```
printf (str-2, 300);
```

```
return 0;
```

```
}
```

```
c) int main()
{
    char str[20] = "Hello";
    char * const p = str;
    *p = 'M';
    printf("%s\n", str);
    return 0;
}
```

OUTPUT } Mello

```
d) int main()
{
    int ***r, **q, *p, i = 8;
    p = &i;
    q = &p;
    r = &q;
    return
    printf("%d, %d, %d\n", *p, **q, ***r);
    return 0;
}
```

OUTPUT 8, 8, 8

```
e) int main()
{
    char * str;
    str = "K";
    printf("%s", str);
    return 0;
}
```

OUTPUT K

```
f) int main()
{
    printf("%d, %d\n", sizeof(NULL),
           sizeof(""));
    return 0;
}
```

OUTPUT 4, 1

```
g) int main()
{
    void *vp;
```

OUTPUT JACK

```
    char ch = 74, *cp = "JACK";
    int j = 65;
    vp = &ch;
    printf("%c", *(char*)vp);
    vp = &j;
    printf("%s", (char*)vp + 2);
    return 0;
}
```

vp & cp w of
same address
*vp = JACK

```
h) int main()
{
```

10	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0

```
    int arr[2][2][2] = {1, 2, 3, 4, 5, 6, 7, 8};
```

```
    int *p, *q;
```

```
    p = &arr[1][1][1];
```

```
    q = (int*)arr;
    printf("%d, %d\n", *p, *q);
    return 0;
}
```

OUTPUT 8, 10

TUT-8 (Cont.)

*r *p

```
printf("%s\n", *r *p);
return 0;
}
```

```
k) int main()
{
```

```
char* str;
str = "ed/n";
```

```
str++;
```

```
str++; (Nothing passed in the pointer)
```

```
printf(str-2, 300);
return 0;
}
```

OUTPUT

300

Aligns character pointed by s2 to location pointed by s1, then advances s1 & s2 to next character

OUTPUT hello

```
o) int main()
{
```

```
char str1[] = "India";
```

```
char str2[] = "BIX";
```

```
char *s1 = str1, *s2 = str2;
```

```
while (*s1++ = *s2++)
```

```
printf("%s", str1);
```

```
return 0;
}
```

```
l) int main()
{
```

```
char str[] = "peace";
```

```
char *s = str;
```

```
printf("%s\n", s++ + 3);
return 0;
}
```

OUTPUT

ce

loop1 :- Bndia loop2 :- Bldia

loop3 :- BIXia (after that str2 would have null terminator thus closing the loop)

OUTPUT Bndia Bldia BIXia

Doesn't print x?

```
p) int main()
{
```

```
int i, n;
```

```
char* x = "Alice";
```

```
n = strlen(x);
```

```
for (i = 0; i <= n; i++)
```

```
{
```

```
printf("%s", x);
```

```
x++;
```

```
}
printf("%s", x);
x++;
```

```
printf("/n", x);
return 0;
}
```

loop1 Alice

loop2 lice

loop3 ice

loop4 ce

loop5 e

OUTPUT

Alice lice ice ce

```
m) int main()
{
```

```
char* p;
```

```
p = "hello";
```

increments the pointer 'x', chopping off the first character

```
}
printf("%s", x);
x++;
```